



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Course

P. R. POTE PATIL

College of Engineering & Management, Amravati.

Year: 2024-25

Semester: IV

Programming Methodology Lab (CS408)

1. VISION & MISSION OF THE INSTITUTE

Vision: -

To flourish as a center of excellence for producing the skilled technocrats and committed human beings.

Mission: -

- To create conducive environment for teaching & learning.

To impart quality education through demanding academic programs.

To enhance career opportunities by exposure to Industries & recent technologies.

To develop professionals with strong ethics and human values for the betterment of

2. VISION & MISSION OF THE DEPARTMENT

To achieve Excellence in Computer Science and Engineering for serving the growing needs of Software industry and Society.

- To create an ambiance that shall foster the growth for developing innovative and
-
- society.

Vision: -

Mission: -

- entrepreneurial skills.
- To identify areas of specialization upon which the department can concentrate, thus promote academic growth and career opportunities.
- To develop human resource with ethical and moral values for overall personality development to serve the society.

3. PO, PSO, PEO

PROGRAM OUTCOMES:

Engineering Graduates will be able to:

PO 1. Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO 2. Problem analysis: Identify, formulate, review research literature, and analyse complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO 3. Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO 4. Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO 5. Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO 6. The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO 7. Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and the need for sustainable development.

PO 8. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO 9. Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO 10. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO 11. Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO 12. Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PEO's

To prepare the Engineering graduates to

PEO 1. Preparation: To prepare graduates to have knowledge of computer science and competency for careers in industry.

PEO 2. Core Competence: To develop problem solving skills in to the graduates that are required to analyse, design and implement solution with the computer knowledge.

PEO 3. Breadth : To nurture the graduates to be an effective team member, build proficiency in soft skills, inculcate multidisciplinary approach and the ability to relate engineering with social context.

PEO 4. Professional and lifelong learning : To inculcate ethical practices, professionalism, and environmental awareness for sustainable development among students with an attitude towards lifelong learning.

PSO:

To prepare the Engineering graduates to

PSO 1 : System Development Apply knowledge of computer science and principals of software design in building the software or hardware projects.

PSO 2 : Problem Solving Ability Use knowledge of various domains to implement innovative solutions to cater the needs of industry and business following ethical values.

GUIDELINES FOR TEACHERS

Teachers shall discuss the following points with students before the start of practicals of the subject.

- 1. Learning Overview:** To develop a better understanding of the importance of the subject.
To know related skills to be developed such as intellectual and motor skills.
- 2. Know your Laboratory Work:** To understand the layout of the laboratory, specifications of equipment/instruments/materials, procedure, working in groups, planning time, etc. also to know total amount of work to be done in the laboratory.
- 3.** The teacher shall ensure that the required equipment is in working condition before the start of each experiment, and also keep the operating instruction manual available.
- 4.** Explain prior concepts to the students before starting each experiment.
- 5.** Evolve student's activity at the time of conduct of each experiment.
- 6.** While taking reading/observation each student (from a batch of 20 students) shall be given a chance to perform/observe the experiment.
- 7.** The teacher shall assess the performance of students continuously.
- 8.** The teacher is expected to share the skills to be developed in the students.
- 9.** The teacher should ensure that the respective skills are developed in the students after the completion of the practical exercise.
- 10.** The teacher may provide additional knowledge and skills to the students even though not covered in the manual but are expected from students by the industries.
- 11.** The teacher may suggest the students to refer additional related literature of the technical papers/reference books / Seminar Proceedings, etc.
- 12.** Focus should be given on the development of enlisted skills rather than theoretical / codified knowledge.
- 13.** During assessment teacher is expected to ask questions to the students to tap their achievements regarding related knowledge and skills. **14.** Teachers should give more focus on hands-on skills.

INSTRUCTIONS FOR STUDENTS

1. Students shall read the points given below for understanding the theoretical concepts and practical applications.
2. Listen carefully to the lecture given by teacher about importance of subject, curriculum philosophy, learning structure, skills to be developed, information about equipment, instruments, procedure, method of continuous assessment, tentative plan of work in laboratory and total amount of works to be done in a semester.
3. Student shall undergo study visit of the laboratory for types of equipment, and material to be used, before performing experiments.
4. Read the write up of each experiment to be performed, a day in advance.
5. Organize the work in the group and make a record of all observations.
6. Understand the purpose of experiment and its practical implications.
7. Student should not hesitate to ask any difficulty faced during conduct of practical.
8. Write the answers of the questions allotted by the teacher during practical hours if possible or afterwards, but immediately.
9. Student should develop the habit of peer discussion / group discussion related to experiments / exercise so that exchanges of knowledge / skills could take place.
10. Students shall attempt to develop related hands-on-skills and gain confidence.
11. Student shall focus on development of skills rather than theoretical or codified knowledge.
12. Student shall insist for the completions of recommended Laboratory Work, answers to the given question etc.
13. Student shall develop the habit of evolving more ideas, innovations, skills etc. that included in the scope of the manual.
14. Student shall refer technical magazines, proceedings of the Seminars, refer website related to the scope of the subjects and update their knowledge and skills.
15. Student should develop the habit of not depend totally on teachers but to develop selflearning techniques.
16. Student should develop the habit to interact with the teacher without hesitation with respect to academic involved.
17. Student should develop habit to submit the practical's exercise continuously and progressively on the schedule dates and should get the assessment done.
18. Student should be well prepared while submitting the write up of the exercise. This will develop the continuity of the studies and he will not be overloaded at the end of the term.

P. R. POTE PATIL
COLLEGE OF ENGINEERING & MANAGEMENT, AMRAVATI.

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Certificate

This is to certify that Mr./Ms..... of
Semester of Bachelor of Technology in **Computer Science & Engineering** of
P. R. Pote Patil College of Engineering & Management, Amravati , has
completed the term work satisfactory of course for the
academic year 2024- 2025 as prescribed in the curriculum.

Date.....

Roll No.....

Practical Incharge

Head of the Department

COURSE OBJECTIVES AND OUTCOMES

Course Objectives

1. To implement the principles of Object-Oriented Programming.
2. To Design and develop GUI applications in Java.

Course Outcomes

On successful completion of the course the learner will be able to:

CO	Course outcomes	BT Level	PO
1	Apply the principles of Object-Oriented Programming.	3	1,12
2	Implement exception handling, file handling and event handling in Java.	3	1,2,3
3	Create GUI applications in Java.	6	1,2,3

LIST OF PRACTICALS & PROGRESSIVE ASSESSMENT FOR TERM WORK

Academic Year:2024-25

Course.....

Course Code:

Semester: II

Name of Faculty:

Name of Student:

Roll No.....

SN	Title of the Practical	Date of Performance	Date of Submission	Sign of Teacher
1	Implement a menu driven Java Program that allows the user to choose an Arithmetic operation on two numbers read from the keyboard.			
2	Implement a Java program to check whether a given number is Armstrong number or not, also find and print the digits of the given number in reverse order.			
3	Implement a Java program to simulates a Banking management system where user can withdraw and deposit money from an account demonstrate the use of encapsulation principle.			

	Dog and Cat that extends Animal and overrides the methods form Animal Class.			
7	Create a program in which you have two rides (Car and Boat) and two locations (Land and Water). Ask from the user about the Ride and the Location. If the user selects ride a car and location water then throw an exception. Similarly, if user selects ride a boat and location land then also throws an exception.			
8	Implement a java program to implement ArrayList using collection framework.			
9	Develop a Java program to Simulate Traffic Signal Using Java Swing Components.			
10	Implement a Java program that handles all mouse events and shows the event name at the center of the window when a mouse event is fired.			
4	Develop a Java program to print details of employees into an organization. Demonstrate the use of parameterized constructors and constructor Signature of Faculty overloading and method overloading.			
5	Demonstrate the concept of inheritance by developing a Java application to generate a pay slip for different categories of employees .			
6	Apply the principal of Run Time Polymorphism by creating a class Animal with methods eat(), Sleep(), makeSound().Create two more classes			

Signature of

Faculty



Rubrics for continuous Assessment

I. Rubrics used for continuous Assessment in every Lab Session

Skills	Allocated Marks	Parameters	High	Medium	Low
Process Related Skills	15	1. Handle equipment/ tools/ commands correctly & safely/Logic Formation (7)	Most satisfactory (4-5) Exceptional (2)	Partially successful (3) Satisfactory (1)	Below expectation (0-2) Unsatisfactory (0)
		1. Integrate system & measure parameters correctly or Debugging Ability (4) 2. Completed experiment as per schedule (4)	Highly satisfactory (4) In time (4)	Partially correct (2-3) *Completed but delayed (2)	Incorrect or unsatisfactory (0-1) *Completed with 50% delayed (1)
Product Related Skills	10	Obtain correct results, Interpret results (4)	Highly Accurate(4)	Partially correct (2-3)	Incorrect (0-1)
		Draw conclusion (3)	Highly accurate (3)	Partially (2)	Unsatisfactory (1)
		Answer practical related questions & submit the write up of experiment on time (3)	Highly satisfactory (3)	Moderate satisfactory (2)	Unsatisfactory (1)
Total Marks		25 Marks			

Skills	Allocated Marks	Parameters	High	Medium	Low
Process Related Skills	15	Handle equipment/ tools/ commands correctly & safely, and Work cohesively in team (7)	Most satisfactory (6-7)	Partially successful (4-5)	Below expectation (0-3)
		Integrate system correctly & measure parameters correctly (8)	Highly satisfactory (7-8)	Partially correct (5-6)	Incorrect or unsatisfactory (0-4)
Product Related Skills	10	Obtain correct results, Answer practical related questions (5)	Exceptional (4-5)	Satisfactory (3)	Unsatisfactory (0-2)
		Interpret results, draw conclusion (5)	Exceptional (4-5)	Satisfactory (3)	Unsatisfactory (0-2)
Total Marks		25 Marks			

**II. Rubrics
External**

used for



Assessment in Lab Course

Assessment

Marks Rubrics: 25 = a (07) + b (04) + c (04) + d (04) + e (03) + f (03)

a: Utilization of S/W b: Debugging c: Execution As Per Schedule d:

Result & Interpretation e: Conclusion f: Write-up & Viva

S.N.	Title of the Practical	(a) 07	(b) 04	(c) 04	(d) 04	(e) 03	(f) 03	Total 25 Marks
1	Implement a menu driven Java Program that allows the user to choose an Arithmetic operation on two numbers read from the keyboard.							

2	Implement a Java program to check whether a given number is Armstrong number or not, also find and print the digits of the given number in reverse order.							
3	Implement a Java program to simulate a Banking management system where user can withdraw and deposit money from an account demonstrate the use of encapsulation principle.							
4	Develop a Java program to print details of employees into an organization. Demonstrate the use of parameterized constructors and constructor Signature of Faculty overloading and method overloading.							
5	Demonstrate the concept of inheritance by developing a Java							

	makeSound().Create two more classes Dog and Cat that extends Animal and overrides the methods form Animal Class.								
7	Create a program in which you have two rides (Car and Boat) and two locations (Land and Water). Ask from the user about the Ride and the Location. If the user selects ride a car and location water then throw an exception. Similarly, if user selects ride a boat and location land then also throws an exception.								
8	Implement a java program to implement ArrayList using collection framework.								
9	Develop a Java program to Simulate Traffic Signal Using Java Swing Components.								
10	Implement a Java program that handles all mouse events and shows the event name at the center of the window when a mouse event is fired.								
	application to generate a pay slip for different categories of employees .								
6	Apply the principal of Run Time Polymorphism by creating a class Animal with methods eat(), Sleep(),								

Signature of

Faculty



PRACTICAL NO. 1

AIM: Develop a java program using Keywords, Identifiers, Operators and Expressions.

OBJECTIVES:

1. Learn and identify Java reserved keywords that have a predefined meaning in the language. These keywords cannot be used as identifiers.
2. Learn the rules and conventions for naming variables, methods, classes, and other entities.
3. Implement various Java operators in expressions and statements.

Keywords are reserved words in Java that have a specific meaning and cannot be used for any other purpose (like naming variables, classes, etc.). Java keywords are predefined by the language and are fundamental to its syntax.

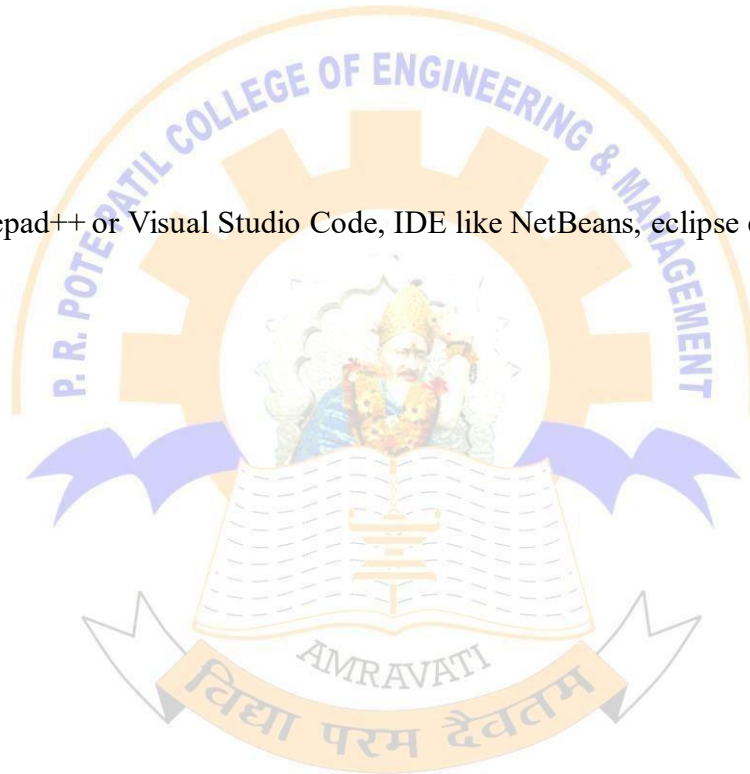
Characteristics of Keywords:

- Keywords are case-sensitive (e.g., public is different from Public).
- They represent actions, control flow, data types, or modifiers.

Common Java Keywords:

- **Control Flow:** if, else, switch, case, for, while, do, break, continue, return, throw, try, catch, finally, throw, import.
- **Data Types:** int, double, char, boolean, String (not technically a keyword but treated as such), void.
- **Access Modifiers:** public, private, protected.
- **Modifiers:** static, final, abstract, synchronized, transient, volatile, native.
- **Other Special Keywords:** new, this, super, instanceof, null, package, class, interface.

Notepad or Notepad++ or Visual Studio Code, IDE like NetBeans, eclipse etc.



4. Develop a Java program that utilizes all the above elements to solve a practical problem.

SOFTWARE REQUIRED:

THEORY:

1) **Java Keywords:**

2) **Java Identifiers:**

Identifiers are the names given to elements like variables, methods, classes, and other entities in Java. Identifiers must follow a set of naming rules:

- The name must begin with a letter (a-z, A-Z), a dollar sign (\$), or an underscore (_).
- Subsequent characters can include letters, digits (0-9), dollar signs (\$), or underscores (_).
- Identifiers are case-sensitive (variable is different from Variable).
- Identifiers cannot be the same as keywords.

3) Java Operators:

Operators are symbols that perform operations on variables or values. Java provides several types of operators:

- Arithmetic Operators: Perform mathematical calculations.
 - Example: +, -, *, /, %
- Relational Operators: Compare two values.
 - Example: ==, !=, >, <, >=, <=
- Logical Operators: Perform logical operations.
 - Example: && (AND), || (OR), ! (NOT)
- Assignment Operators: Assign values to variables.
 - Example: =, +=, -=, *=, /=
- Unary Operators: Operate on a single operand.
 - Example: ++ (increment), -- (decrement)

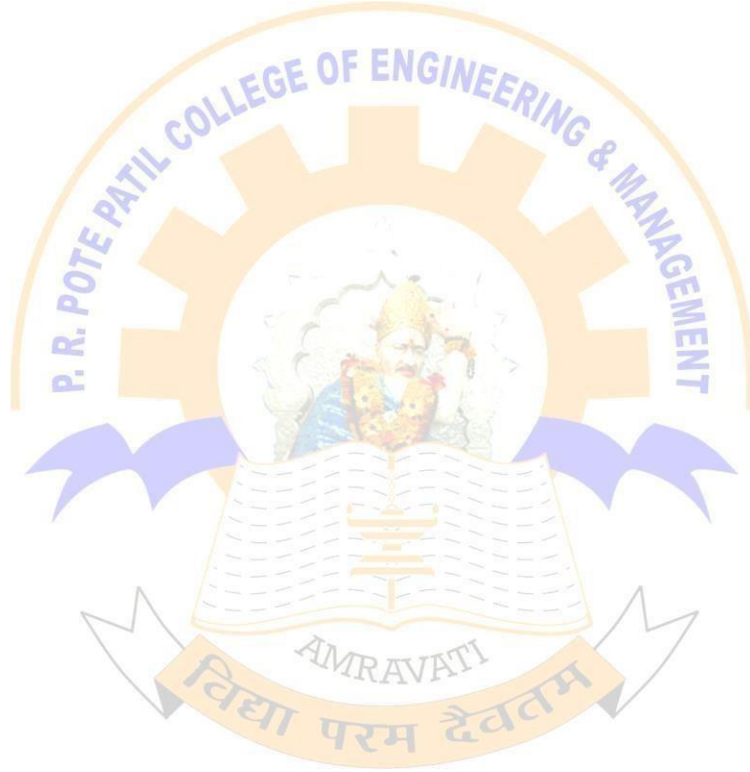
Java Expressions:**4)**

An expression is a combination of operators, variables, constants, and method calls that evaluate to a single value. Expressions are used to perform calculations or comparisons in Java. They can be simple (e.g., $a + b$) or complex (e.g., $((x + y) * z) / 2$).

PROGRAM:

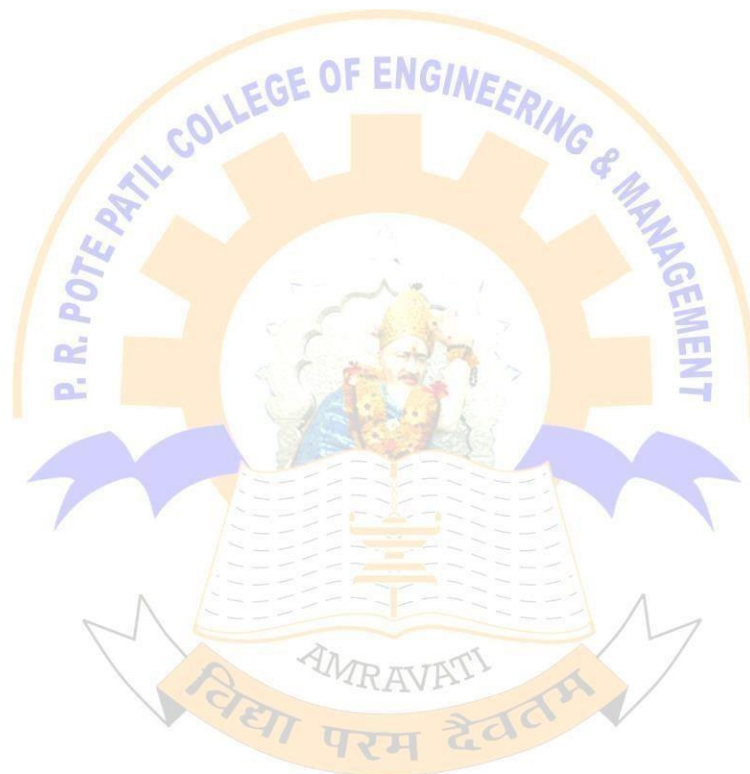
Implement a menu driven Java Program that allows the user to choose an Arithmetic operation on two numbers read from the keyboard.

SOURCE CODE:





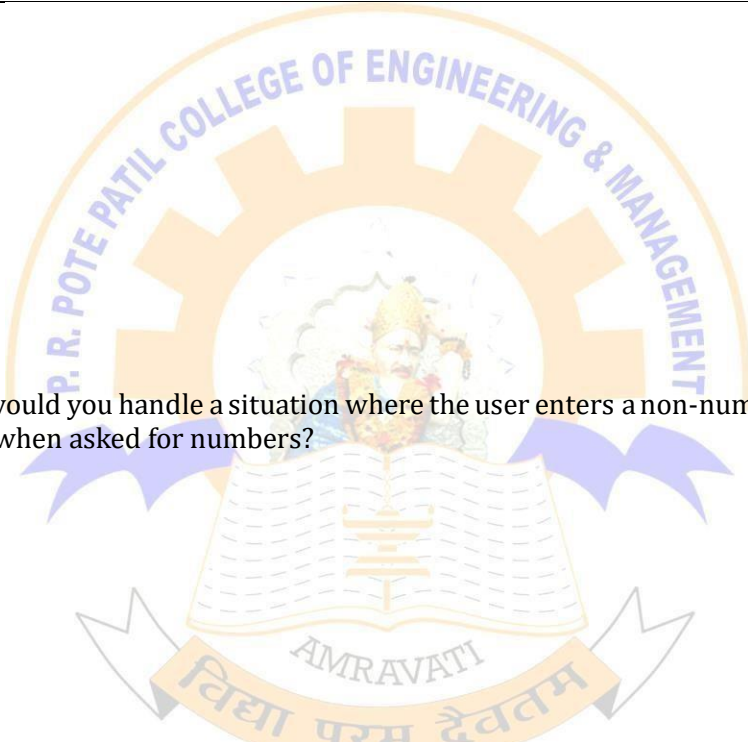
OUTPUT:



CONCLUSION:

--

LAB QUESTIONS:

Sr No.	Questions
1	What is the purpose of using a Scanner class in Java?
2	What is the significance of the switch statement in Java?
3	How would you handle a situation where the user enters a non-numeric value when asked for numbers? 

ASSESSMENT SCHEME:

Process Related Skills (15-M)	Product Related Skills (10-M)	Total (25-M)	Signature of Faculty
--	--	---------------------	---------------------------------

--	--	--	--

PRACTICAL NO. 2

AIM: Implementation of conditional, looping and branching statements using Java Programming.

OBJECTIVES:

In Java programming, conditional, looping, and branching statements are fundamental concepts that control the flow of execution in a program.

1. Conditional Statements

Conditional statements allow the program to execute different code based on specific conditions. The most commonly used conditional statements in Java are:

if Statement: The if statement is used to evaluate a condition and execute a block of code if the condition is true. <pre>if (condition) { // Code to be executed if condition is true }</pre>	if-else Statement: The if-else statement is used when there are two possible actions, depending on whether the condition is true or false. <pre>if (condition) { // Code to be executed if condition is true } else { // Code to be executed if condition is false }</pre>
--	--

To enhance the control the program based conditions. By else, switch, do-while branching program can repeat actions, complex logic helps in dynamic and applications.



ability to flow of a on specific utilizing if, while, for, and loops, and constructs, the make decisions, and manage efficiently. This developing interactive

SOFTWARE

Notepad or Notepad++ or Visual Studio Code, IDE like NetBeans, eclipse etc.

REQUIRED:

THEORY:

else-if Ladder: The else-if ladder allows you to check multiple conditions sequentially. If the first condition is false, the next condition is checked, and so on.

```
if(condition1) {
    // Code to be executed if condition1 is true
} else if(condition2) {
    // Code to be executed if condition2 is true } else {
// Code to be executed if none of the
conditions are true
}
```

switch Statement: The switch statement is an alternative to multiple if-else conditions when there are many possible choices for a variable. It is typically used for checking the value of a variable against different constant values.

```
switch
    (express
ion)  {
    case
value1:
        // Code to be executed
if expression equals value1
break;
    case value2:
        // Code to be executed
if expression equals value2
break;
    default:
        // Code to be executed
if no case matches
}
```

2. Looping Statements

Looping statements are used to repeat a block of code multiple times. In Java, there are three main types of loops:

□ for Loop: The for loop is used when the number of iterations is known beforehand. It consists of three parts: initialization, condition, and iteration. <pre>for (int i = 0; i < 10; i++) { // Code to be executed repeatedly }</pre>	□ while Loop: The while loop is used when the number of iterations is not known beforehand. It executes as long as a specified condition is true. <pre>while (condition) { // Code to be executed repeatedly }</pre>
--	--

□ do-while Loop: The do-while loop is similar to the while loop, but it guarantees that the code block will be executed at least once, as the condition is checked after the loop's body is executed. <pre>do { // Code to be executed repeatedly } while (condition);</pre>	
--	--

Branching statements control the flow of execution and can be used to exit or jump to a

break Statement: The break statement is used to exit from a loop or switch statement prematurely, stopping its execution.

```
for (int i = 0; i <
    10; i++) {
    if(i == 5) {
        break; // Exit the loop when i equals 5
```

continue Statement: The continue statement is used to skip the current iteration of a loop and move to the next iteration.

3. Branching Statements

different part of the program.

□

```
}}
```

□

```
for (int i = 0; i
<10; i++) {
    if(i==5) {
        continue; // Skip the rest of the code for i equals 5 }
    // Other code to be executed
}
```

□ **return Statement:** The return statement is used to exit from a method and optionally return a value. It can also be used to exit a method prematurely.

```
public int sum(int a, int b) { return a + b; // Exits the  
method and returns the sum}
```



PROGRAM:

Implement a Java program to check whether a given number is Armstrong number or not, also find and print the digits of the given number in reverse order.

SOURCE CODE:





OUTPUT:

CONCLUSION:

LAB QUESTIONS:

Process Related Skills (15-M)	Product Related Skills (10-M)	Total (25-M)	Signature of Faculty

Sr No.	Questions
1	Explain how your Java program determines whether a number is an Armstrong number.
2	How does the program reverse the digits of the number?
3	What changes or improvements could you make to your Java program?

ASSESSMENT SCHEME:

PRACTICAL NO. 3

AIM: Implementation of Encapsulation in Java Programming.

OBJECTIVES:

- To implement encapsulation in Java by hiding the internal data of a class and

restricting direct access to it, ensuring that the data is accessed only through well defined methods.

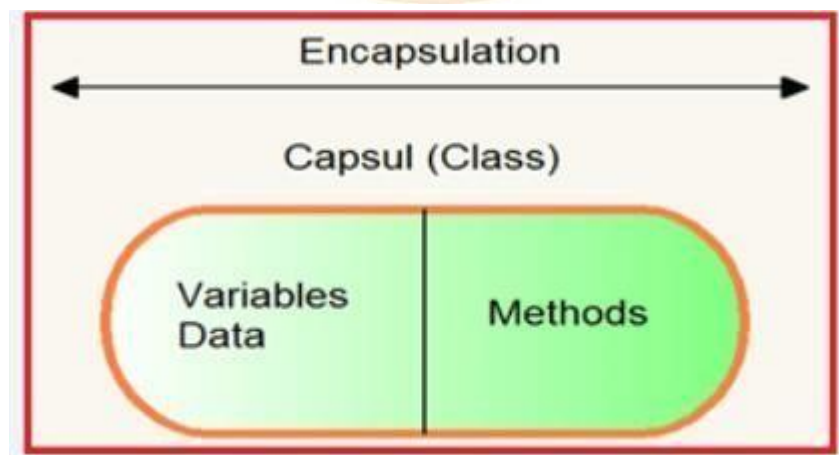
- To promote code reusability and maintainability by separating the internal implementation details of a class from its external interface

SOFTWARE REQUIRED:

Notepad or Notepad++ or Visual Studio Code, IDE like NetBeans, eclipse etc. .

THEORY:

Encapsulation is one of the fundamental principles of Object -Oriented Programming (OOP), which refers to the concept of bundling the data (variables) and methods (functions) that operate on the data into a single unit or class. In encapsulation, the internal representation of an object is hidden from the outside world, and access to the data is provided only through well-defined methods, known as **getters** and **setters**. This concept is also referred to as **data hiding**.



1. Private Variables (Data Hiding):

- The internal data (variables) of a class are declared **private**, meaning they cannot be accessed directly from outside the class. This is the essence of data hiding.
- For example, in a BankAccount class, the account balance is private to prevent unauthorized external changes.

2. Public Methods (Accessors and Mutators):

- To allow controlled access to the private variables, **public methods** are provided. These methods are known as **getters** (accessors) and **setters** (mutators).
- **Getter methods** return the value of the private variables.
- **Setter methods** allow the value of the private variables to be modified, often after validating the input to ensure correctness.

3. Benefits of Encapsulation:

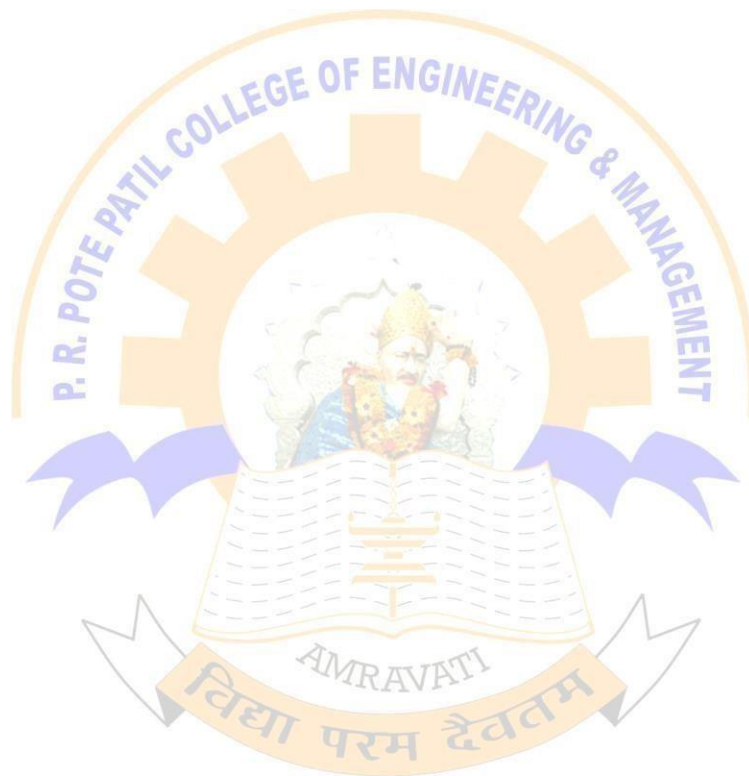
- **Data Security:** By making the data private, you can control how and when it can be accessed or modified, preventing unauthorized changes or misuse.
- **Increased Flexibility:** Since the internal workings are hidden, you can change the internal code (e.g., how data is stored or validated) without affecting other parts of the program that rely on the object.
- **Code Maintainability:** Encapsulation makes it easier to maintain and modify code because the details are encapsulated in classes, and external code interacts with the class via a defined interface (methods).
- **Avoids Unintended Side Effects:** Since the object's data is accessed and modified through methods, you can ensure that the object's behavior remains consistent, even if changes are made to the internal logic.

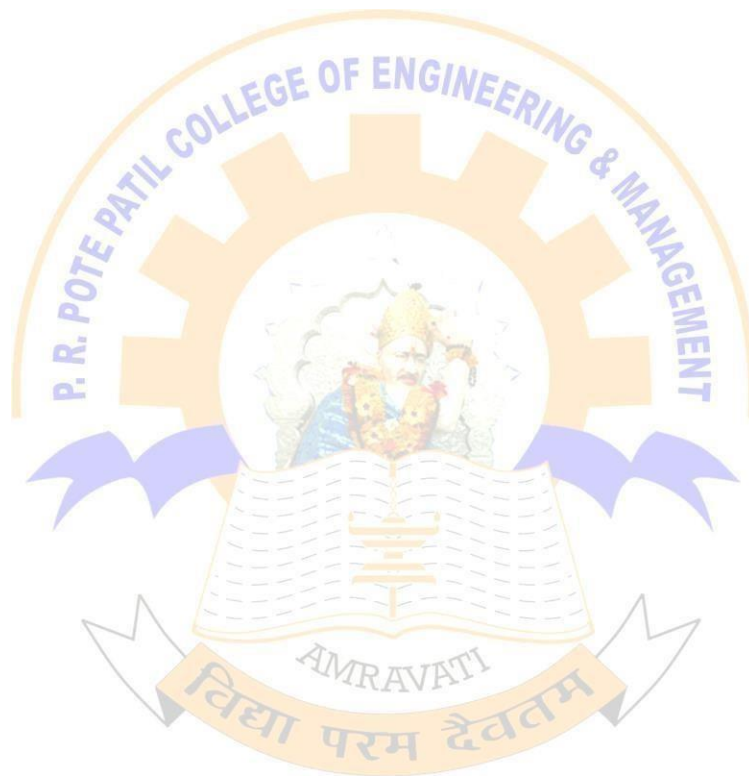
PROGRAM:

Implement a Java program to simulate a Banking management system where user can withdraw and deposit money from an account demonstrate the use of encapsulation principle.

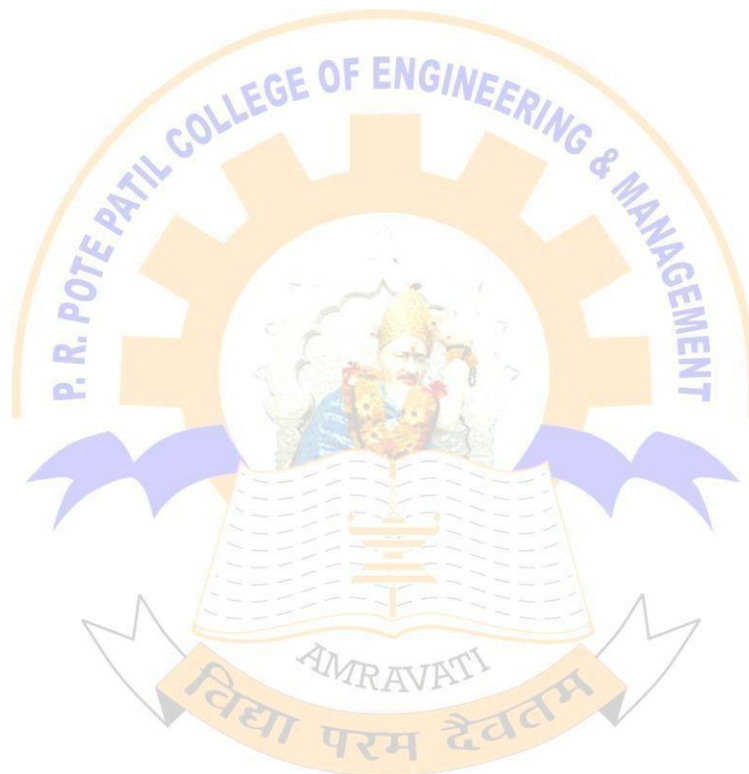
SOURCE CODE:





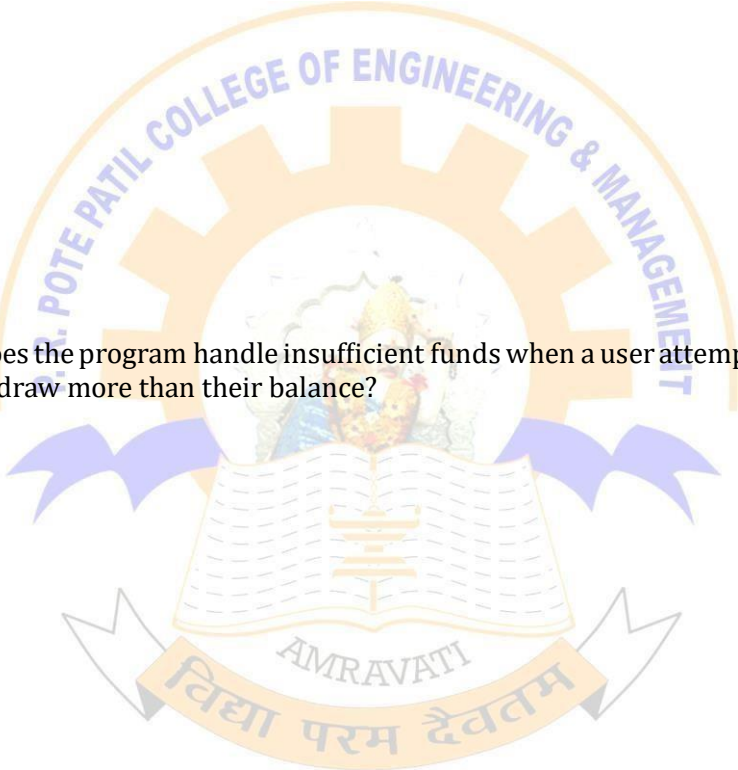


OUTPUT:



CONCLUSION:

LAB QUESTIONS:

Sr No.	Questions
1	How does your program implement encapsulation?
2	Describe how the deposit and withdrawal methods work in your program.
3	How does the program handle insufficient funds when a user attempts to withdraw more than their balance? 

ASSESSMENT SCHEME:

Process Related Skills (15-M)	Product Related Skills (10-M)	Total (25-M)	Signature of Faculty
----------------------------------	----------------------------------	--------------	----------------------

--	--	--	--

PRACTICAL NO. 4

AIM: Perform Java program to demonstrate uses of constructors.

OBJECTIVES:

1. Learn the fundamental concept of constructors in Java and their primary purpose, which is to initialize objects when they are created.
2. Understand the difference between the two main types of constructors: **default constructors** (which take no parameters) and **parameterized constructors**
3. Learn about **constructor overloading**, which allows the definition of multiple constructors with different parameter lists to provide flexibility in how objects are initialized.

SOFTWARE REQUIRED:

Notepad or Notepad++ or Visual Studio Code, IDE like NetBeans, eclipse etc. .

THEORY:**1. What are Constructors in Java?**

In Java, a Constructor is a block of codes similar to the method. It is called when an instance of the class is created. At the time of calling the constructor, memory for the object is allocated in the memory. It is a special type of method that is used to initialize the object. Every time an object is created using the new() keyword, at least one constructor is called.

How Java Constructors are Different From Java Methods?

- Constructors must have the same name as the class within which it is defined it is not necessary for the method in Java.
- Constructors do not return any type while method(s) have the return type or void if does not return any value.
- Constructors are called only once at the time of Object creation while method(s) can be called any number of times.

Need of Constructors in Java

Think of a Box. If we talk about a box class then it will have some class variables (say length, breadth, and height). But when it comes to creating its object(i.e Box will now exist in the computer's memory), then can a box be there with no value defined for its dimensions? The answer is **No**.

So constructors are used to assign values to the class variables at the time of object creation, either explicitly done by the programmer or by Java itself (default constructor).

2) Types of Constructors in Java

Now is the correct time to discuss the types of the constructor, so primarily there are three types of constructors in Java are mentioned below:

- Default Constructor
- Parameterized Constructor
- Copy Constructor

1. Default Constructor in Java

A constructor that has no parameters is known as default constructor.

Implicit Default Constructor: If no constructor is defined in a class, the Java compiler automatically provides a default constructor. This constructor doesn't take any parameters and initializes the object with default values, such as 0 for numbers, null for objects.

2. Parameterized Constructor in Java

A constructor that has parameters is known as parameterized constructor. If we want to initialize fields of the class with our own values, then use a parameterized constructor.

3. Copy Constructor in Java

Unlike other constructors copy constructor is passed with another object which copies the data available from the passed object to the newly created object.

```
public class Main {  
    int x; public  
    Main() {  
        System.out.println("I am Default constructor");  
    }  
  
    public Main(int y) {  
        x = y;  
        System.out.println("I am Parameterized constructor");  
    } public static void main(String[]  
args) {  
        Main myObj = new Main();  
        Main myObj1 = new Main(5);  
        System.out.println(myObj1.x);  
    }  
}
```

1) Constructor Overloading:

Constructor overloading is the practice of defining multiple constructors in a class, each having a different number or types of parameters. This allows objects to be initialized in different ways, depending on the constructor invoked.

When do we need Constructor Overloading?

Sometimes there is a need of initializing an object in different ways. This can be done using constructor overloading.

Example:-

```
public class Box {
    double width, height, depth;
    int boxNo;

    Box(double w, double h, double d, int num)
    {
        width = w;
        height = h;
        depth = d;
        boxNo = num;
    }

    Box()
    {
        // an empty box
        width = height = depth = 0;
    }

    Box(int num)
    {
        this();
        boxNo = num;
    }

    public static void main(String[] args)
    {
        Box box1 = new Box(1);
        System.out.println(box1.width);
    }
}
```

PROGRAM:

Develop a Java program to print details of employees of an organization .Demonstrate the use of parameterized constructors and constructor overloading and method overloading.

SOURCE CODE:





OUTPUT:

```
Default Constructor Called
Name: Unknown, Age: 0, Department: Not Assigned, Salary: 0.0
Parameterized Constructor Called
Name: Alice, Age: 30, Department: IT, Salary: 75000.0
Partial Details Constructor Called
Partial Details - Name: Bob, Age: 0, Department: HR, Salary: 0.0
Employee Details - Name: Alice, Age: 30, Department: IT, Salary: 75000.0
PS C:\Users\Dhiraj\OneDrive\Desktop\IV Sem\java>
```

CONCLUSION:

LAB QUESTIONS:

Sr No.	Questions
1	What is the purpose of parameterized constructors in the Employee class?
2	Explain Constructor Overloading in the context of your Employee class.
3	Real-world Application of Constructor Overloading and Method Overloading.

ASSESSMENT SCHEME:

Process Related Skills (15-M)	Product Related Skills (10-M)	Total (25-M)	Signature of Faculty

PRACTICAL NO. 5

AIM: Demonstrate various types of inheritance using Java Program.

OBJECTIVES:

To demonstrate and understand the concept of inheritance in Java by implementing various types of inheritance, including single inheritance, multilevel inheritance, hierarchical inheritance, and hybrid inheritance.

SOFTWARE REQUIRED:

Notepad or Notepad++ or Visual Studio Code, IDE like NetBeans, eclipse etc.

THEORY:

Inheritance is one of the fundamental concepts in Object-Oriented Programming (OOP), which allows a class to inherit properties and behaviors (fields and methods) from another class. In Java, inheritance promotes code reusability and establishes a relationship between classes. It enables a class to acquire the functionality of another class while adding its own specific features. Java supports multiple types of inheritance, each serving different use cases and scenarios. Below are the primary types of inheritance in Java:

1. Single Inheritance

Single inheritance is the simplest form of inheritance where one class (subclass) inherits the properties and methods from a single parent class (superclass).

```
class Animal {  
    void eat() {  
        System.out.println("This animal eats food.");  
    }  
}
```

```
class Dog extends Animal { void  
    bark() {  
        System.out.println("The dog  
barks."); } }
```

```
public class TestInheritance { public static  
    void main(String[] args) { Dog dog =  
        new Dog(); dog.eat(); // Inherited  
        method dog.bark(); // Method specific  
        to Dog }
```

```
}
```

2. Multilevel Inheritance

Multilevel inheritance occurs when a class inherits from a class that is also a subclass of another class. This forms a chain of inheritance. **Example:**

```
class Animal { void
    eat() {
        System.out.println("This animal eats food.");
    }
}
```

```
class Dog extends Animal { void
    bark() {
        System.out.println("The dog
barks."); } }
```

```
class Puppy extends Dog {
    void play() {
        System.out.println("The puppy
plays."); } }
```

```
public class TestInheritance { public static
    void main(String[] args) { Puppy puppy =
    new Puppy(); puppy.eat(); // Inherited from
    Animal puppy.bark(); // Inherited from Dog
    puppy.play(); // Method specific to Puppy
    }
}
```

3. Hierarchical Inheritance

Hierarchical inheritance occurs when multiple classes inherit from a single parent class. This allows different subclasses to share common functionality from the same parent class.

Example:

```
class Animal { void
    eat() {
        System.out.println("This animal eats food.");
    }
}
```

```
class Dog extends Animal { void
    bark() {
        System.out.println("The dog
barks."); } }
```

```
class Cat extends Animal { void
    meow() {
        System.out.println("The cat
meows."); } }
```

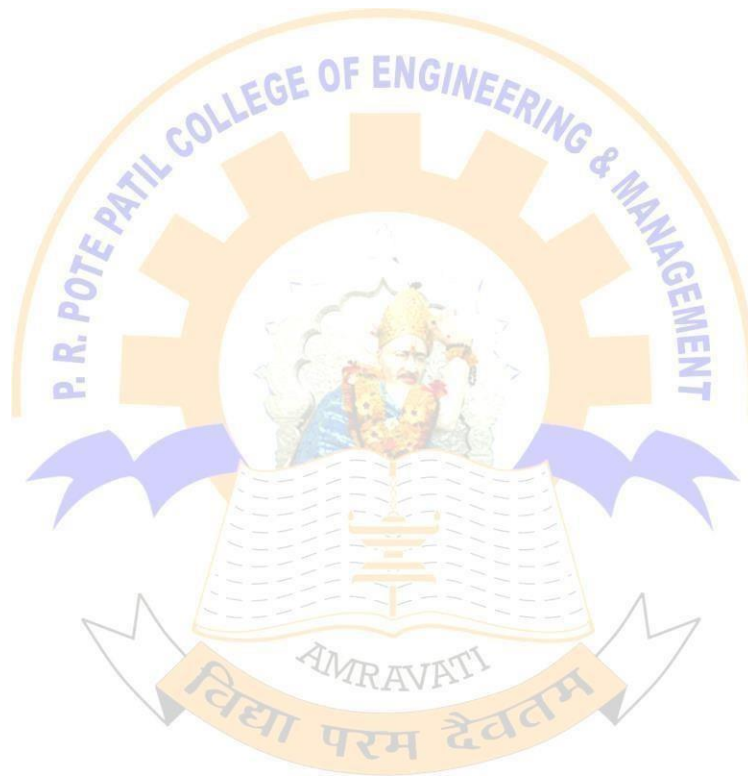
```
public class TestInheritance { public
    static void main(String[] args) {
        Dog dog = new Dog();
    }
}
```

```
dog.eat(); // Inherited from Animal dog.bark();  
// Specific to Dog  
  
Cat cat = new Cat(); cat.eat(); //  
Inherited from Animal cat.meow();  
} // Specific to Cat }
```

PROGRAM:

Demonstrate the concept of inheritance by developing a Java application to generate a pay slip for different categories of employees .

SOURCE CODE:





OUTPUT:

```
--- Pay Slip for Programmer ---
Employee ID: 101
Name: Alice
Basic Pay: 50000.0
DA (25%): 12500.0
HRA (10%): 5000.0
PF (12%): 6000.0
Gross Salary: 67500.0
Net Salary: 61500.0

--- Pay Slip for Professor ---
Employee ID: 201
Name: Dr. Bob
Basic Pay: 60000.0
DA (30%): 18000.0
HRA (15%): 9000.0
PF (12%): 7200.0
Gross Salary: 87000.0
Net Salary: 79800.0
PS C:\Users\Dhiraj\OneDrive\Desktop\IV Sem\java>
```

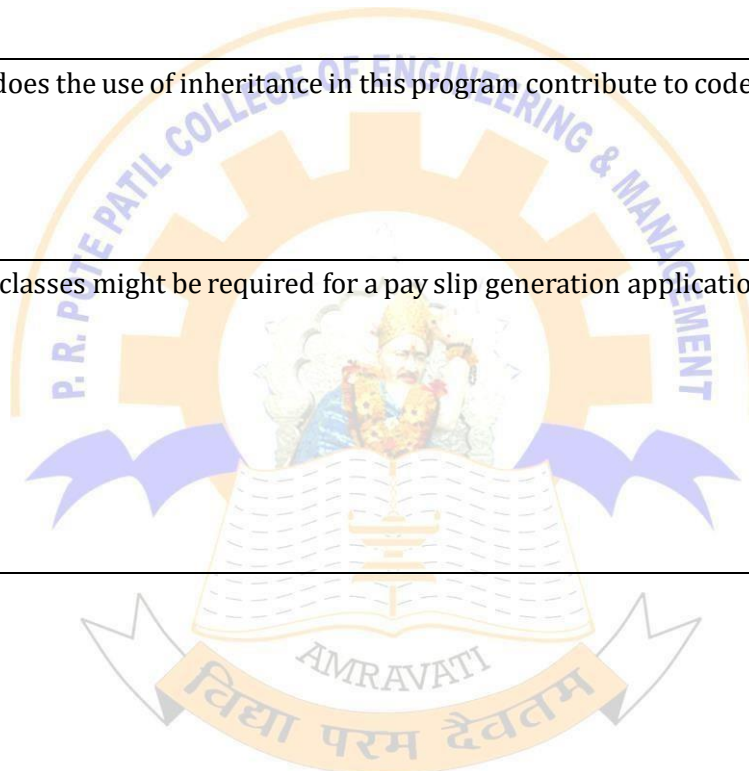
CONCLUSION:

ASSESSMENT SCHEME:

Process Related Skills (15-M)	Product Related Skills (10-M)	Total (25-M)	Signature of Faculty

LAB QUESTIONS:

Sr No.	Questions
1	How does inheritance help in creating a flexible and maintainable payroll system?
2	How does the use of inheritance in this program contribute to code reuse?
3	What classes might be required for a pay slip generation application?



PRACTICAL NO. 6

AIM: Develop Java Program to Demonstrate polymorphism.

OBJECTIVES:

- To demonstrate runtime polymorphism (also known as method overriding), where a subclass provides its specific implementation of a method already defined in its superclass.
- to promote the writing of more flexible, reusable, and maintainable Java programs that can handle different data types and objects effectively.

SOFTWARE REQUIRED:

Notepad or Notepad++ or Visual Studio Code, IDE like NetBeans, eclipse etc. .

THEORY:

Polymorphism in Java

Polymorphism is one of the four fundamental Object-Oriented Programming (OOP) concepts, along with encapsulation, inheritance, and abstraction. In simple terms, polymorphism allows objects of different classes to be treated as objects of a common superclass. The term "polymorphism" comes from the Greek words "poly" (meaning many) and "morph" (meaning form), so it means "many forms."

In Java, polymorphism allows a method to behave differently based on the object that calls it. There are two types of polymorphism in Java:

1. **Compile-Time Polymorphism (Method Overloading):** This occurs when multiple methods with the same name are defined, but they differ in the number or type of parameters. The method to be invoked is determined at compile-time based on the method signature.
2. **Run-Time Polymorphism (Method Overriding):** This occurs when a subclass provides a specific implementation of a method that is already defined in its superclass. The method to be invoked is determined at runtime based on the object's actual class type, not the reference type.

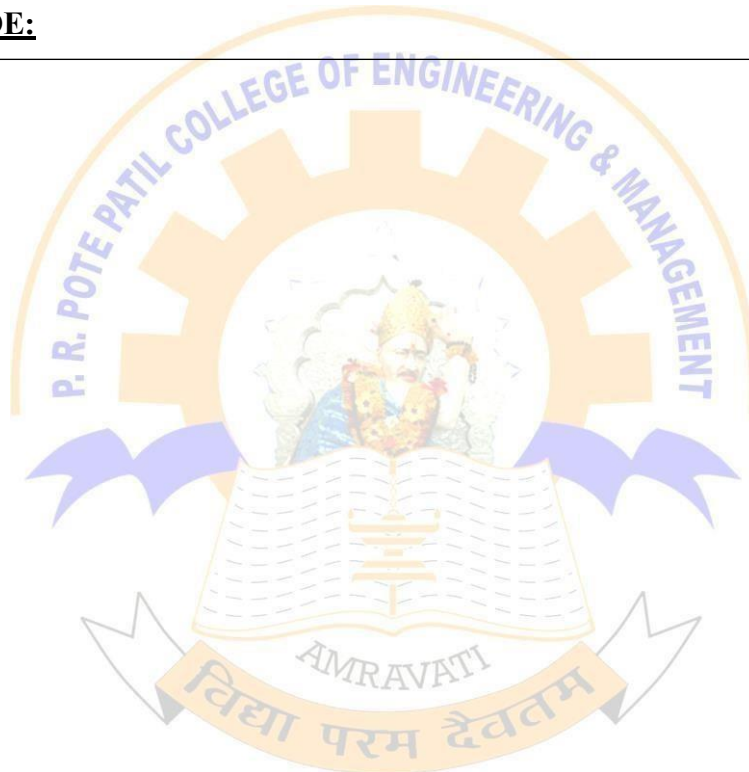
Why Polymorphism is Important:

- **Flexibility:** Polymorphism allows for writing more flexible and reusable code. You can create a generic code structure that works with different types of objects.
- **Maintainability:** Code becomes easier to maintain and extend since new classes can be introduced without modifying the existing code that uses polymorphism.

PROGRAM:

Apply the principal of Run Time Polymorphism by creating a class Animal with methods eat(), Sleep(), makeSound(). Create two more classes Dog and Cat that extends Animal and overrides the methods from Animal Class.

SOURCE CODE:



OUTPUT:

```
Dog's Behavior:
Dog is eating bones...
Dog is sleeping in its kennel...
Dog says: Woof! Woof!

Cat's Behavior:
Cat is eating fish...
Cat is sleeping on the couch...
Cat says: Meow! Meow!
PS C:\Users\Dhiraj\OneDrive\Desktop\IV Sem\java>
```

CONCLUSION:

--

LAB QUESTIONS:

Sr No.	Questions
1	How does your program implement Polymorphism?
2	How did method overriding contribute to achieving runtime polymorphism in the program?
3	What would happen if method overriding wasn't implemented in Dog and cat?

ASSESSMENT SCHEME:

Process Related Skills (15-M)	Product Related Skills (10-M)	Total (25-M)	Signature of Faculty

PRACTICAL NO. 7

AIM: Develop Java program to demonstrate Exceptional Handling Mechanism.

OBJECTIVES:

1. Learn how to handle both checked and unchecked exceptions.
2. Learn the foundational concepts of exceptions in Java, such as try, catch, throw, throws, and finally blocks.
3. Learn how to create custom exception classes..

In Java, **exception handling** is a mechanism to handle runtime errors, ensuring the normal flow of the application. Java provides a robust way to handle exceptions using a combination of keywords and classes. Exceptions are events that disrupt the normal flow of the program, such as division by zero, trying to access a null object, or attempting to open a file that does not exist.

Java handles exceptions by allowing the programmer to define a block of code that might cause an exception and specify how to deal with it. This is accomplished using the try, catch, throw, throws, and finally keywords.

Exception Types in Java

Java categorizes exceptions into two main types:

A. Checked Exceptions:

These exceptions are checked at compile time. The compiler requires that you either handle these exceptions using a try-catch block or declare them in the method signature using throws.

Example: IOException, SQLException.

B. Unchecked Exceptions:

These exceptions are not checked at compile time and typically represent programming bugs (e.g., accessing an array out of bounds or dividing by zero).

Example: ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException.

SOFTWARE REQUIRED:

Notepad or Notepad++ or Visual Studio Code, IDE like NetBeans, eclipse etc. .

THEORY:

1. What is Exception Handling in Java?

2. Components of Exception Handling

1. Try Block:

- The try block is used to enclose the code that might throw an exception. If an exception occurs in this block, the control is transferred to the corresponding catch block.

```
try {  
    // Code that might throw an exception  
}
```

2. Catch Block:

- The catch block is used to handle the exception thrown by the try block. You can have multiple catch blocks to handle different types of exceptions.

```
catch (ExceptionType e) {  
    // Code to handle the exception  
}
```

3. Throw Statement:

- The throw statement is used to explicitly throw an exception from a method or a block of code. This is useful for creating custom exceptions or rethrowing exceptions for further processing.

```
throw new ExceptionType("Message");
```

4. Throws Keyword:

- The throws keyword is used in the method signature to declare that a method may throw one or more exceptions. It allows the caller of the method to handle the exception.

```
public void someMethod() throws ExceptionType  
{ // Code that might throw an exception }
```

5. Finally Block:

- The finally block is used to execute code that must run regardless of whether an exception is thrown or not. This block is generally used for clean-up code, such as closing resources (e.g., files, database connections).

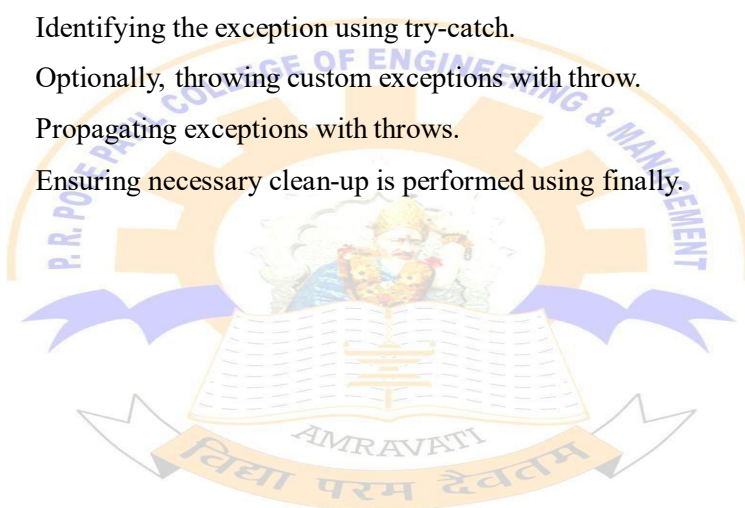
```
finally {  
    // Clean-up code, always executed  
}
```

3) How Exception Handling Works in Java

When an exception occurs, the Java Virtual Machine (JVM) searches for an appropriate catch block to handle it. If no catch block is found in the current method, the exception is propagated to the calling method, and this process continues up the call stack until it is caught or the stack is exhausted. If the exception is not caught, the program terminates with an error message.

The process of handling an exception involves:

- Identifying the exception using try-catch.
- Optionally, throwing custom exceptions with throw.
- Propagating exceptions with throws.
- Ensuring necessary clean-up is performed using finally.



PROGRAM:

Create a program in which you have two rides (Car and Boat) and two locations (Land and Water). Ask from the user about the Ride and the Location. If the user selects ride a car and location water then throw an exception. Similarly, if user selects ride a boat and location land then also throws an exception.

SOURCE CODE:





OUTPUT:

```
Enter ride (Car or Boat): boat
Enter location (Land or Water): water
You selected a boat on water. Ride is allowed.
PS C:\Users\Dhiraj\OneDrive\Desktop\IV Sem\java>
```

CONCLUSION:

LAB QUESTIONS:

Sr No.	Questions
1	What is the role of the try-catch block in Java?
2	How does the program handle invalid input combinations?
3	What would happen if the user enters an unrecognized value for the ride or location? What would happen if the try-catch block were removed from the program?

ASSESSMENT SCHEME:

Process Related Skills (15-M)	Product Related Skills (10-M)	Total (25-M)	Signature of Faculty

PRACTICAL NO. 8

AIM: Develop Java Program to implement data structures using collection framework.

OBJECTIVES:

1. To Understand the Basics of Java Collection Framework.

The **Java Collection Framework** is a set of classes and interfaces that implement various types of collections, such as lists, sets, and maps. The main objective of the collection framework is to provide a unified architecture for representing and manipulating collections of objects. It includes interfaces, classes, and algorithms that make it easier to handle collections of data in an efficient and organized way.

Java provides several **data structures** as part of the collection framework, allowing developers to store and manipulate data. These data structures can be broadly categorized into the following types:

1. **List:** An ordered collection that can contain duplicate elements.
2. **Set:** A collection that does not allow duplicate elements.
3. **Queue:** A collection designed for holding elements before processing them, typically in a FIFO (First-In-First-Out) order.
4. **Map:** A collection that stores key-value pairs, where each key is unique.

Benefits of Using the Collection Framework

1. **Flexibility:** The collection framework allows developers to choose from various data structures (e.g., ArrayList, HashSet, HashMap) based on their needs, such as whether they need ordered elements, unique elements, or key-value pairs.
2. **Efficiency:** The framework provides optimized implementations of various data structures, ensuring that developers don't have to implement their own complex data structures.
3. **Automatic Resizing:** Data structures like ArrayList and HashMap automatically resize themselves when more space is needed, so developers don't need to worry about capacity management.

2. Learn how to implement dynamic arrays and linked lists using the List interface, including classes such as ArrayList and LinkedList.

SOFTWARE REQUIRED:

Notepad or Notepad++ or Visual Studio Code, IDE like NetBeans, eclipse etc.

THEORY:

1. Java Collection Framework

Operations on Data Structures

Each collection in the Java Collection Framework supports basic operations, which are crucial for manipulating the data stored in these structures. Common operations include:

1. Adding Elements: Insert new elements into the collection.

- In List: add(E e)
- In Set: add(E e)
- In Map: put(K key, V value)

2. Removing Elements: Delete elements from the collection.

- In List: remove(Object o)
- In Set: remove(Object o)
- In Map: remove(Object key)

3. Searching for Elements:

- In List: contains(Object o) checks if the list contains a particular element.
- In Set: contains(Object o) checks if the set contains a particular element.
- In Map: containsKey(Object key) and containsValue(Object value) check if a key or value exists in the map.

4. Iterating Over Elements:

- You can iterate over a collection using an Iterator, for-each loop, or Java Streams.
- In List, Set, and Map, you can use forEach(), iterator(), or enhanced for loops for traversal.

5. Accessing Elements:

- In List: get(int index) to retrieve an element by its index.
- In Map: get(Object key) to retrieve the value associated with a specific key.

PROGRAM:

Implement a java program to implement ArrayList using collection framework.

SOURCE CODE:



OUTPUT:

```
Initial List: [Banana, Pineapple, Apple]
Enter an item to add: gauva
After adding 'gauva': [Banana, Pineapple, Apple, gauva]
Item at index 1: Pineapple
After setting index 1 to 'Mango': [Banana, Mango, Apple, gauva]
After removing 'Apple': [Banana, Mango, gauva]
Enter item to check: gauva
gauva is in the list.
Size of the list: 3
All items in the list:
- Banana
- Mango
- gauva
PS C:\Users\Dhiraj\OneDrive\Desktop\IV Sem\java>
```

CONCLUSION:

LAB QUESTIONS:

Sr No.	Questions
1	What is the underlying data structure of the ArrayList class? How does it differ from an array?
2	Explain how you implemented the methods add(), remove(), get(), and size() in your ArrayList class. Are they consistent with the built-in ArrayList methods? Why or why not?
3	What happens if you try to access an element at an index that is out of bounds in an ArrayList?

ASSESSMENT SCHEME:

Process Related Skills (15-M)	Product Related Skills (10-M)	Total (25-M)	Signature of Faculty

PRACTICAL NO. 9

AIM: Design GUI based Java applications using swing.

OBJECTIVES:

1. Learn understanding of the Java Swing library, its components, and how it is used for building graphical user interfaces (GUIs) in Java.
2. To Design and implement a basic Swing application that showcases how to set up a GUI window and add simple components.

SOFTWARE REQUIRED:

Notepad or Notepad++ or Visual Studio Code, IDE like NetBeans, eclipse etc.

Java Swing

Java Swing is a part of Java's standard library, specifically designed for building graphical user interfaces (GUIs). It is an API that allows developers to create rich, interactive desktop applications with graphical components like windows, buttons, text fields, labels, and more. Swing is built on top of the Abstract Window Toolkit (AWT), but it is more powerful and flexible. Unlike AWT, Swing provides a lightweight component set, meaning it doesn't rely on the native OS components but rather uses its own set of UI elements, making it highly customizable and cross-platform.

Swing is **platform-independent**, meaning applications built using Swing will run consistently across different operating systems like Windows, macOS, and Linux.

Core Components of Swing

1. **JFrame:** This is the main window or the frame in which all other components are placed. It serves as the container for your GUI components.
 - **Example:** `JFrame frame = new JFrame("Calculator");`
2. **JPanel:** A generic container for organizing components. It can hold other components and is used to group them for layout management.
 - **Example:** `JPanel panel = new JPanel();`
3. **JButton:** A button component that allows the user to trigger actions by clicking. It can display text or images.
 - **Example:** `JButton button = new JButton("Click Me");`
4. **JTextField:** A single-line text box where users can input text. It's often used for user input in forms or for displaying output.
 - **Example:** `JTextField textField = new JTextField();`
5. **JLabel:** A label to display text or images. It's typically used to display information or instructions to the user.

2. Java

THEORY:

Collection Framework



- **Example:** JLabel label = new JLabel("Enter your name:");
- 6. **JComboBox:** A drop-down list that allows users to select a value from a list of options.
 - **Example:** JComboBox comboBox = new JComboBox(options);
- 7. **JList:** A list of items, allowing users to select one or more items from a list.
 - **Example:** JList<String> list = new JList<>(items);
- 8. **JMenu:** Used for creating menus (like "File", "Edit", etc.). Menus can contain different menu items and submenus.
 - **Example:** JMenu menu = new JMenu("File");

Swing Layout Managers

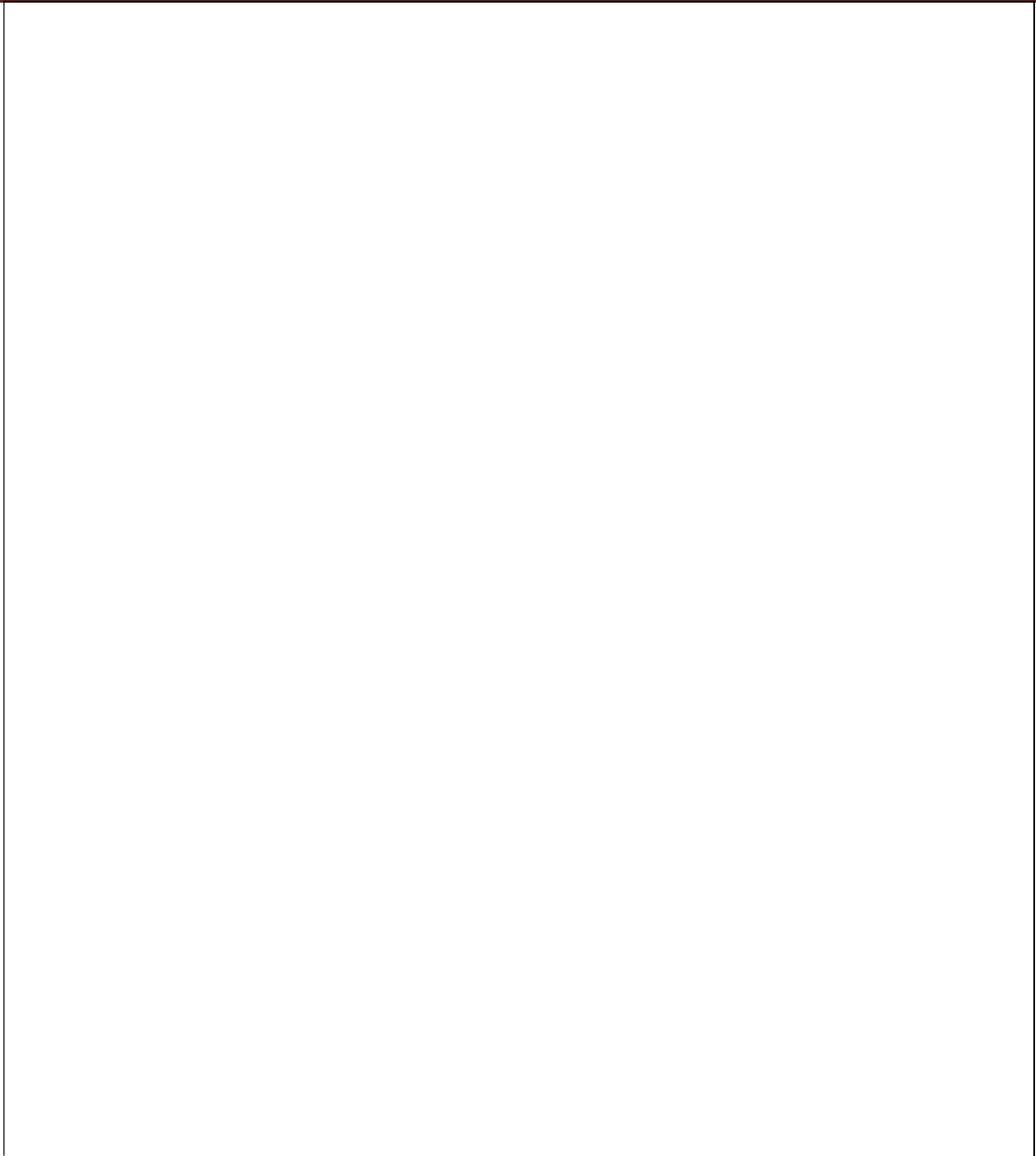
Layout managers are used to control the arrangement of components within a container. Swing provides several types of layout managers, such as:

1. **FlowLayout:** Components are added one after another, from left to right, and top to bottom. When the container's width is exceeded, components wrap to the next line. ○ Example: FlowLayout layout = new FlowLayout();
2. **BorderLayout:** Divides the container into five regions: North, South, East, West, and Center. Each component is placed in one of these regions. ○ Example: BorderLayout layout = new BorderLayout();
3. **GridLayout:** Arranges components in a grid with a specified number of rows and columns.
 - Example: GridLayout layout = new GridLayout(3, 2); (3 rows, 2 columns)
4. **BoxLayout:** Organizes components either vertically or horizontally.
 - Example: BoxLayout layout = new BoxLayout(container, BoxLayout.Y_AXIS);
5. **GridBagLayout:** A more complex layout that allows components to span multiple rows or columns, providing fine-grained control over placement.
 - Example: GridBagLayout layout = new GridBagLayout();

PROGRAM:

Develop a Java program to Simulate Traffic Signal Using Java Swing Components.

SOURCE CODE:







OUTPUT:



CONCLUSION:

--

LAB QUESTIONS:

Process Related Skills (15-M)	Product Related Skills (10-M)	Total (25-M)	Signature of Faculty

Sr No.	Questions
1	How does the layout manager (GridLayout) contribute to the design of the traffic signal simulation?
2	How did you handle the events triggered by the startStopButton and Timer in your program?
3	How could you enhance the user interface of this program?

ASSESSMENT SCHEME:

PRACTICAL NO. 10

AIM: Implement Event Handling in Java Applications.

OBJECTIVES:

1. Learn Understand the Event-Driven Programming Model .
2. Learn About Java Event Classes and Event Sources .
3. To Handle Button Click Events Using ActionListener.

SOFTWARE REQUIRED:

Notepad or Notepad++ or Visual Studio Code, IDE like NetBeans, eclipse etc.

developing graphical user interfaces (GUIs). Event-driven programming allows an application to respond dynamically to user actions such as button clicks, mouse movements, key presses, and window interactions. In Java, event handling is achieved using a well-defined model that allows components to communicate with the application's logic through event listeners.

Event-Driven Programming Model

Event-driven programming in Java relies on the concept of events—actions or occurrences detected by the application—triggered by user input or other sources. In this model, the flow of the program is determined by the events rather than a sequential set of instructions. When an event occurs (e.g., a user clicks a button), the associated event listener is notified and executes the corresponding code. Java uses the event delegation model, which means that when an event occurs in a component (like a button click), the event is sent to the event source (the component), and it is handled by an event listener (a class that implements a listener interface).

Java Event Handling Mechanism

Java provides a robust event-handling framework for managing events in GUI-based applications. The process involves three main components:

1. **Event Source:** The object (e.g., a button, text field, or window) that generates the event.
2. **Event Object:** The object that contains information about the event (e.g., which button was clicked, which key was pressed).
3. **Event Listener:** The object that listens for and handles the event. It implements a specific event listener interface like `ActionListener`, `MouseListener`, etc.

Event Listener Interfaces

Java provides several built-in listener interfaces, each designed to handle specific types of events. Some of the most common event listener interfaces include:

1. **ActionListener:** Used to handle action events, typically from buttons or menu items. ○
Method: `void actionPerformed(ActionEvent e)`

Department of Computer Science & Engineering

THEORY:

Event

Event
aspect
Java
when



**Handling in Java
Applications**

handling is a crucial
of creating interactive
applications, especially

- Example: Handling a button click event.
- 2. **MouseListener:** Used to handle mouse events such as clicks, presses, releases, etc.
 - Methods: mouseClicked(MouseEvent e), mousePressed(MouseEvent e), mouseReleased(MouseEvent e), mouseEntered(MouseEvent e), mouseExited(MouseEvent e)
 - Example: Handling mouse clicks on a component.
- 3. **KeyListener:** Used to handle keyboard events such as key presses and releases. ○ Methods: keyPressed(KeyEvent e), keyReleased(KeyEvent e), keyTyped(KeyEvent e)
 - Example: Handling key presses in a text field.
- 4. **WindowListener:** Used to handle window events, such as opening, closing, or resizing windows.
 - Methods: windowOpened(WindowEvent e), windowClosing(WindowEvent e), windowClosed(WindowEvent e), etc.
 - Example: Handling the event when a window is closed.
- 5. **FocusListener:** Used to handle focus events when components gain or lose focus.
 - Methods: focusGained(FocusEvent e), focusLost(FocusEvent e)
 - Example: Detecting when a text field gains focus.

PROGRAM:

Implement a Java program that handles all mouse events and shows the event name at the center of the window when a mouse event is fired.

SOURCE CODE:





OUTPUT:



CONCLUSION:

--

LAB QUESTIONS:

Process Related Skills (15-M)	Product Related Skills (10-M)	Total (25-M)	Signature of Faculty

Sr No.	Questions
1	How did you implement the handling of mouse events using the MouseListener and MouseMotionListener.
2	How did you center the event name in the window? What role does FontMetrics play in this process?
3	Why is it necessary to call repaint() after a mouse event occurs?

ASSESSMENT SCHEME: